Gabriel Riven Wahnich

Mobile: 0488 992 283
Email: riven.development@gmail.com | Melbourne VIC
GitHub: github.com/Riiiviii | Portfolio: rivenwahnich.dev
LinkedIn: linkedin.com/in/gabriel-riven-wahnich

PROFILE

Computer Science graduate (Distinction, RMIT) focused on AI engineering and full-stack systems. Building agentic AI applications using the OpenAI Agents SDK and Model Context Protocol, backed by full-stack experience across Python, TypeScript, and PostgreSQL. Recently shipped a containerised agentic chatbot service to production with multi-process orchestration and structured error handling.

TECHNICAL SKILLS

AI Engineering: OpenAI Agents SDK, Model Context Protocol (MCP), FastMCP
Languages: Python, TypeScript, JavaScript, SQL
Frontend: React, Next.js, TanStack Start, TanStack Router, TanStack Query, Tailwind CSS, Zod, Vite
Backend: FastAPI, Pydantic, NestJS, Node.js, REST APIs, JWT Auth
Data & Infrastructure: PostgreSQL, Prisma, Neon, Docker, Fly.io
Tooling: Git, GitHub

EDUCATION

Bachelor of Computer Science with Distinction
Minor – Artificial Intelligence and Machine Learning
RMIT University
December 2025

WORK EXPERIENCE

Freelance Software Engineer | Olways Pets | Melbourne **Jan 2026 - Present**

OlwaysPets is a Melbourne-based commission artist offering handmade pet portraits shipped worldwide. Built a full-stack platform to display work and manage orders.

- Designed and built a full-stack platform for a commission-based pet portrait business, managing ~500 artwork assets, testimonials, and customer enquiries
- Modelled the relational schema in PostgreSQL and exposed it through a NestJS REST API, separating public catalogue endpoints from authenticated admin operations
- Secured the admin panel with JWT authentication and Argon2 password hashing, isolating administrative workflows from public-facing routes
- Offloaded image storage and delivery to Cloudinary, using URL-based transformations to resize and compress assets dynamically reducing backend load and improving frontend performance.
- Built a responsive frontend with React (Vite), Tailwind CSS, shadcn/ui, and Framer Motion.

Software Engineer Internship | Musical Moon | Melbourne **Jan 2025 - May 2025**

Musical Moon is an early-stage startup building a marketplace platform for musician collaboration. Contributed as a developer across both web and mobile.

- Designed and shipped an order history feature end-to-end across web (Next.js) and mobile (Expo), from database schema through to UI

- Built and maintained the order history microservice, handling order data flow between web and mobile clients
- Integrated secure file-download functionality into the Order History interface for cross-artist file sharing
- Containerised the service with Docker, improving environment consistency and simplifying local setup

Systems Assembler and Hardware Technician | PLE Computers | Melbourne **Jun 2021 - Jan 2024**

- Diagnosed hardware faults, assembled custom builds, and applied firmware updates across consumer and enterprise systems.

PROJECTS

AI Portfolio Chatbot | github.com/Riiiviii/ai-chatbot | riven-portfolio-api.fly.dev/docs

- Built an agentic AI service answering questions about my professional background using the OpenAI Agents SDK and a custom FastMCP server exposing seven structured resume tools.
- Containerised the FastAPI app and FastMCP server in a single image, deployed to Fly.io. A bash entry point starts both processes, waits for the MCP server to be ready via a TCP probe, and forwards SIGTERM so the container shuts down cleanly.
- Implemented Pydantic-validated request/response schemas, structured error handling with traceback logging, and environment-driven CORS configuration via a hand-rolled Settings class.

Vestly | github.com/Riiiviii/vestly | vestly-nine.vercel.app/

- Built and deployed a multi-agent financial analysis pipeline that runs four specialist agents (Fundamentals, Sentiment, Risk, Competition) in parallel and a Judge agent that synthesises their findings into an evidence-grounded investment thesis. FastAPI backend on Fly.io, React (Vite) frontend on Vercel.
- Architected the agents on the OpenAI Agents SDK with Pydantic-validated structured outputs, building each panel agent on a shared AnalysisAgent base class and a research-pack data layer drawn from yfinance and Finnhub.
- Gave the Competition agent its own MCP server (FastMCP over stdio) exposing peer-lookup and research-pack tools, so the agent autonomously selects 2–3 relevant competitors via Finnhub and fetches comparison data on demand, turning a single-company analysis into a grounded competitive one.

ADDITIONAL INFORMATION

Languages – English (fluent), Spanish (fluent)

INTERESTS

- Game Development: Building small-scale interactive projects with Godot

REFERENCES

Available upon request